

Coding Standard – Sistema de Reserva de Livros

Objetivo

Este documento define os padrões de codificação adotados no desenvolvimento do Sistema de **Coding Standard – Sistema de Reserva de Livros**

Objetivo

Este documento define os padrões de codificação adotados no desenvolvimento do Sistema de Reserva de Livros, com o objetivo de garantir organização, legibilidade, manutenção e padronização do código-fonte entre os integrantes da equipe.

1. Padrões Gerais

- O código deve ser escrito de forma clara, legível e organizada;
- Deve-se evitar duplicação de código sempre que possível;
- Métodos e funções devem possuir responsabilidade única;
- Comentários devem ser utilizados apenas quando realmente necessários;
- O projeto deve seguir organização modular e separação por responsabilidades;
- Todo código implementado deve estar versionado utilizando Git.

2. Convenções de Nomenclatura

Classes

Utilizar nomes descritivos e em PascalCase.

- **Exemplos:**
 - Livro
 - ReservaService
 - UsuarioController

Métodos e Funções

Utilizar verbos que representem a ação executada.

Utilizar PascalCase ou camelCase conforme a linguagem adotada.

- **Exemplos:**
 - realizarReserva()
 - buscarLivros()
 - cancelarReserva()

Variáveis

Devem possuir nomes claros e objetivos.

Evitar abreviações sem necessidade.

- **Exemplos:**
 - livroDisponivel
 - dataReserva
 - usuarioLogado

3. Organização do Projeto

O sistema deverá ser organizado em camadas, separando responsabilidades entre:

- Apresentação
- Negócio/Serviços
- Persistência de Dados

Cada módulo deve possuir responsabilidade bem definida.

4. Controle de Versão

- O versionamento será realizado utilizando Git;
- Cada funcionalidade deve ser desenvolvida em branch própria;
- Commits devem possuir mensagens claras e objetivas.
- **Exemplos:**
 - feat: adicionar reserva de livros
 - fix: corrigir validação de login
 - refactor: reorganizar serviço de empréstimos

5. Boas Práticas

- Validar entradas de usuários antes do processamento;
- Evitar código duplicado;
- Tratar exceções e erros adequadamente;
- Manter padronização de identificação e formatação;
- Priorizar reutilização de componentes e funções.

6. Testes e Qualidade

- Funcionalidades implementadas devem ser testadas antes da integração;
- Correções devem preservar funcionalidades existentes;
- O código deve manter compatibilidade com a arquitetura definida no projeto.